Smart-X IoT Ecosystem

Project Completion Report and Technology Recommendations

Student	Christopher Graham
Student number	ST10440190
Module	PROG7312 Programming 3B
Institution	Emeris College Newlands
Repository	https://github.com/BIGchrxs/SmartX.Core.PROG7312

---

1. What was built

Smart-X is an IoT gateway for Studio HQ, an interior design and architecture studio. It monitors environmental conditions, power draw and automated actuators across four zones, and it is delivered as one application with three pillars reached from a single menu.

Pillar	Part	What it does
Sensor Data Ingestion and Telemetry	1	Device registration, typed telemetry ingestion, jagged array buffers, media attachment
Real-Time Command Stream and History	2	Queued ingestion with a priority lane, manual overrides with undo, alerts, device timelines, a suggestion engine
Network Topology and Mesh Routing	3	The mesh as a weighted graph, traversals, minimum spanning trees, routing, and an AVL archive for historical queries

The backend is an ASP.NET Core Minimal API on .NET 10. The frontend is a React and TypeScript single page application. A Node script stands in for the ESP32 hardware and sends ordinary HTTP requests, so every structure is exercised the way it would be in the field rather than through a test harness.

Data structures across the three parts

Structure	Where	Why it was chosen
`Queue<QueuedReading>`	`IngestionQueue`	Normal readings must be processed in arrival order
`PriorityQueue<QueuedReading, (int, long)>`	`IngestionQueue`	A critical reading must bypass the backlog
`Stack<CommandEntry>`	`CommandService`	Undo reverses the most recent command first
`Dictionary<string, SensorRegistration>`	`SensorRegistry`	Constant time device lookup, case insensitive
`Dictionary<string, string>`	`SensorRegistry`	Constant time lookup by hardware address
`SortedDictionary<DateTime, HistoryPoint>`	`TelemetryHistory`	The live timeline is always in time order
`HashSet<string>`	`AlertTracker`	One alert per problem, duplicates impossible by construction
`AvlTree<DateTime, HistoryPoint>`	`TelemetryArchive`	Range queries over a long history in O(log n + k)
Adjacency list	`MeshNetwork`	A sparse mesh; O(V + E) memory instead of O(V squared)
`PriorityQueue<MeshLink, int>`	`MeshNetwork.PrimMst`	The cheapest frontier link in O(log E)
`DisjointSet`	`MeshNetwork.KruskalMst`	“Already connected?” in near constant time

How the result was verified

Correctness is measured by the application rather than asserted in prose. Three cross checks run on real data and are visible in the interface:

- **Two algorithms, one answer.** Prim’s and Kruskal’s build the spanning tree from opposite directions. On the Studio HQ

mesh both return 8 links at a total cost of 25.

- **Tree against scan.** Every range query is answered twice, once down the AVL tree and once by scanning an unsorted list of identical readings. The results are compared. On an 8,013 reading archive the tree visited 250 nodes where the scan examined all 8,013, and both returned the same 240 readings.
- **Balanced against unbalanced.** The same 2,000 readings are built into both an AVL tree and a plain binary search tree. The AVL tree is 11 levels deep; the plain tree is 2,000.

Alongside this, 135 automated checks pass: 46 against the Part 3 API, 46 driving the Part 3 page in a real browser, 29 covering Parts 1 and 2, and 14 covering the Part 2 panels.

---

2. Architectural and algorithmic challenges

2.1 Choosing a tree, after noticing how telemetry arrives

The brief allows a binary search tree, an AVL tree or a red black tree. The deciding factor turned out to be not the query but the **order the data arrives in**.

Telemetry is keyed by timestamp, and timestamps only ever increase. For an unbalanced binary search tree that is the worst possible input: every new reading compares greater than the current node and goes down the right hand side, so the tree never branches. What is meant to be an O(log n) structure becomes a linked list with O(n) lookup, and it would be slower than the `List` it was supposed to replace while using more memory.

Rather than take that on trust, I built the measurement into the application. `BalanceProof` inserts the same readings into both trees and reports the heights. Over 2,000 readings the AVL tree is 11 levels deep after 1,989 rotations, and the unbalanced tree is 2,000 levels deep. Finding the newest reading costs 11 comparisons against 2,000.

That number is also why `Height()` on the comparison tree is written as a loop with an explicit stack rather than as recursion. A recursive walk of a 2,000 level tree risks a stack overflow, and on a longer history it would be a certainty. The structure being measured dictated how the measurement itself had to be written.

What was gained: a data structure is only as good as its fit with the access pattern *and* the insertion pattern. The second is easy to overlook and, here, it was the one that mattered.

2.2 Range queries that do not walk the whole tree

An in order traversal returns readings in time order, but it visits every node. The saving in a range query comes from pruning: at each node, two comparisons decide whether either side can hold a match at all, and a subtree that cannot is skipped whole.

Getting the conditions right took care. The left child is only worth visiting when the current node is already past the start of the range, and the right child only when the current node is still before the end. An early version used the inclusion test for both and silently walked branches it did not need, which was not visible in the results, only in the node count.

That is exactly why the node count is surfaced in the API response and on the page. A wrong answer announces itself; wasted work does not, unless it is measured.

2.3 Minimum spanning tree and shortest path are not the same question

The first routing implementation used the spanning tree, which the brief asks for. Checking it against Dijkstra’s algorithm revealed a discrepancy: `MS-DEHUM-01` reaches the gateway for a cost of 9 on its own cheapest path, but for 12 along the spanning tree.

The initial reading was that something was broken. It was not. A minimum spanning tree minimises the power of the **whole network**, by keeping 8 links awake instead of 14. Dijkstra minimises the cost for **one node**. Both are correct answers to different questions, and the 3 unit difference is what network wide efficiency costs that particular node.

The honest response was to show both and explain the trade off in the interface, rather than pick one and hide the tension. A dashboard that showed only the spanning tree route would quietly mislead an operator asking why one sensor’s battery drains faster than expected.

2.4 Asynchronous navigation that does not destroy state

The Part 3 rubric asks for asynchronous navigation, and the Part 2 requirement to never lose application state still stands. These pull against each other. The standard React approach, `React.lazy` with `Suspense`, defers loading a route until it is opened, but the usual pattern of rendering only the active route unmounts the previous one and takes its state with it.

The resolution was to separate **loading** from **mounting**. The shell keeps a list of every route opened so far. A route enters that list the first time it is visited, its bundle is fetched at that moment, and from then on it stays mounted and is merely hidden when another route is active.

```
const [opened, setOpened] = useState<Route[]>(() => [route]);

useEffect(() => {
  setOpened((previous) => (previous.includes(route) ? previous : [...previous, route]));
}, [route]);
```

The build confirms the split is real: each pillar compiles to its own chunk, and the browser test asserts that the topology bundle is requested only on first visit.

2.5 Simulated hardware that behaved like a race condition

While driving the Part 2 dashboard with browser automation, a command to switch an actuator on would occasionally revert a second later. The API was correct. The fault was in the simulator, which read every actuator's state once per cycle and then sent all readings from that snapshot. A command landing mid cycle was overwritten by an echo of the state from before it.

Real hardware reads its own pin at the moment it reports, so the fix was to make the simulator do the same and read each actuator immediately before sending it.

What was gained: the value of a test that drives the real interface. No unit test of the API would have found this, because the API was never wrong. Only exercising the whole system the way a user does exposed it.

2.6 Precision in what the interface says

A warning message read "above the warning limit" when the value was exactly equal to the limit. The threshold check used `>=` and was correct; the sentence describing it was not.

It is a one word fix, but it is the kind of inaccuracy that erodes trust in a monitoring tool. The wording is now "at or above the limit", which matches the comparison the code performs.

2.7 Delivery is part of engineering, not an afterthought

The most costly problem in this project had nothing to do with algorithms.

Part 2 was written, tested and committed locally across 41 commits, but those commits were never pushed to GitHub. What the marker could see was the Part 1 code. The feedback reflects that precisely: "A queue supports path validation, not the required telemetry ingestion and critical-alert processing. No command undo stack found." That queue is `DeviceNode.IsValidPath`, a Part 1 helper. It was an accurate assessment of what had actually been delivered.

The root cause was a branch per feature workflow used without a merge step. Work sat on five feature branches that were never merged into `master`, and the folder open in the editor was on a documentation branch that contained none of the implementation. Locally everything ran, which is exactly what made the gap invisible.

The corrective steps were to merge every feature branch, push, and then verify from a position that could not be fooled by local state: clone the repository fresh into a new folder and run it from there. That clean clone test is now the last step before any submission is considered done.

What was gained: "it works on my machine" and "it is delivered" are different claims, and only the second one is marked. Verification has to be done from the position of the person receiving the work, not the person who produced it.

---

3. Key learnings

Choose structures from how data arrives, not only from how it is read. The AVL decision turned on monotonic timestamps. The same reasoning drove the priority queue key `(-priority, sequence)`, where the sequence number preserves arrival order between readings of equal priority, and the `HashSet` for alerts, which makes a duplicate impossible rather than merely checked for.

Measure instead of claiming. Every performance statement in this project is produced by the running application: nodes visited against items scanned, tree height against ideal height, heap operations, rotations, Prim against Kruskal. Building the measurement in cost little and it caught real problems, including the unpruned range query.

Cross check with an independent method. Two algorithms agreeing is much stronger evidence than one algorithm not crashing. This applied to Prim and Kruskal, to the tree and the scan, and in testing to comparing the spanning tree cost against 300 randomly built spanning trees.

Test through the interface people actually use. The simulator race condition and a layout overflow at phone width were both found by automation driving a real browser, and neither would have appeared in a unit test.

Finish the delivery. The single largest mark loss in this project came from work that existed and functioned but had not reached the repository. That lesson cost more than any algorithm in it.

---

4. Technology recommendations and future scaling

These are ordered by what would most improve the system next.

4.1 GitHub Actions for continuous integration

What: a workflow that runs on every push: `dotnet build`, `dotnet test`, `npm ci`, `npm run build`, and the browser suite against a started API.

Why first. This project's largest failure was delivering a repository whose contents did not match the working code. A pipeline that builds from a clean checkout would have failed immediately and visibly, because the pushed branches did not contain the Part 2 files at all. Every other recommendation here improves a system that works; this one protects against shipping one that does not.

Fit with .NET. `actions/setup-dotnet` and `actions/setup-node` are first party, and the browser suite already written for this project runs unchanged on a GitHub runner. The repository is already on GitHub, so there is nothing to migrate.

4.2 A time series database for the archive

What: TimescaleDB, a PostgreSQL extension, or Azure Data Explorer for the telemetry archive.

Why. The AVL tree is the right structure for range queries in memory, and the measurements in this report show it working. It has two limits that are properties of the design rather than faults in it: everything is lost when the process stops, and the history must fit in the memory of one process. A hypertable in TimescaleDB answers the same range queries on disk, partitioned by time, and survives a restart.

Fit with .NET. Npgsql is the mature PostgreSQL driver for .NET and Entity Framework Core supports it fully. Because the archive is already behind the `TelemetryArchive` class, swapping the storage means changing one class rather than touching the endpoints or the dashboard.

The honest trade off. This adds a dependency and a network hop to a query that is currently measured in ticks. It is worth it when history must outlive a restart, and not before.

4.3 Azure IoT Hub with the Device Provisioning Service

What: replace the direct HTTP ingestion endpoint with Azure IoT Hub for device connectivity.

Why. `SensorRegistry` and `MacAddressPolicy` are a hand built device registry: identity, hardware address, and a check that a device is known before its readings are accepted. IoT Hub provides that as a managed service, and adds what a real deployment needs and this one does not have: per device credentials with X.509 certificates, MQTT so battery powered ESP32 nodes can hold one cheap connection instead of opening an HTTP request per reading, and the ability to revoke one device without touching the others.

Fit with .NET. The Azure IoT SDK for .NET is first party, and ESP-IDF on the ESP32 side has a supported client. The Device Provisioning Service matches the pattern Smart-X already uses, where a device announces itself and is enrolled.

4.4 Docker Compose, then container orchestration

What: a `Dockerfile` for the API, one for the dashboard, and a `docker-compose.yml` that starts both with the database from 4.2.

Why. Running the system currently takes three terminals and a correct SDK on the machine. One `docker compose up` removes every version difference between machines, which is the same class of problem as the clean clone test in 4.1: it proves the repository contains everything needed to run. Beyond that, container images are the unit that orchestration platforms schedule, so this is the step that makes horizontal scaling possible later.

Fit with .NET. Microsoft publishes official `dotnet/sdk` and `dotnet/aspnet` images, and the multi stage build pattern of compiling with the SDK image and shipping on the smaller runtime image is well established.

4.5 OpenTelemetry with Prometheus and Grafana

What: export the counters the application already keeps as real metrics.

Why. Smart-X already measures heap operations, AVL rotations, tree nodes visited, queue depth and how many readings a critical reading bypassed. Today these are returned in API responses and read by a human. Exported as metrics they become history: a graph of queue depth over a week would show whether the priority lane is ever under genuine pressure, and tree height over time would show the archive growing.

Fit with .NET. `System.Diagnostics.Metrics` is part of the base library, and the OpenTelemetry .NET SDK exports from it with a few lines in `Program.cs`. The instrumentation points already exist; only the export is missing.

4.6 SignalR in place of polling

What: push updates to the dashboard over a WebSocket instead of polling every two seconds.

Why. The dashboard currently asks the API for everything on a timer, whether anything changed or not. That is simple and it works for one operator, but the request count grows with the number of open dashboards rather than with the number of events. Pushing would also remove the deliberate 350 millisecond delay after a command, which exists only to let the background worker finish before the next poll.

Fit with .NET. SignalR is part of ASP.NET Core, and `@microsoft/signalr` is the matching client. Because all shared state already lives in one React context, the change is confined to that context.

4.7 An LLM API for a daily telemetry summary

What: a scheduled job that sends the day's alerts, commands and learned habits to a language model and stores a short written summary.

Why. The suggestion engine answers "what should I do now". It does not answer "what happened yesterday", which is the question asked in a Monday morning handover. The data for that already exists in `ActivityLog` and `AlertTracker` and is currently only read live.

Fit and caution. A hosted model is reached over plain HTTPS from a background service, so it fits the existing worker pattern. The qualification matters more than the integration: a generated summary is a convenience layer over the records, not a replacement for them, and it should link back to the alerts it describes so an operator can check it. Telemetry leaving the building for a third party service is also a decision to take deliberately rather than by default.

4.8 Home Assistant integration

What: expose Smart-X devices to Home Assistant, the open source home and building automation platform, through MQTT discovery.

Why. Studio HQ is a working building. Home Assistant already speaks to a very wide range of commercial hardware, so an integration would let Smart-X control equipment it has no driver for, while keeping its own queueing, alerting and routing logic. It also brings mobile notifications and scheduling without building either.

Fit with .NET. MQTT discovery is a documented message format rather than a library dependency, so the API publishes a configuration message per device and then publishes state. `MQTTnet` is the established MQTT client for .NET. This pairs naturally with 4.3, since both move the system onto MQTT.

---

5. Closing

The Smart-X project is complete across all three parts, with every pillar live inside one application and every required data structure in service of a real feature rather than added to satisfy a checklist.

The work that most changed how I approach a project was not an algorithm. It was learning, at a cost, that engineering includes delivering the work in a form the recipient can actually run, and that this must be verified from their position rather than from mine.

Christopher Graham | ST10440190 | Emeris College Newlands